

Lecture Notes Template

A visual specimen for CS 351

Dr. A. Belcaid

Table of contents

1 Purpose **1**
1.1 Mathematical typography 2
1.2 Code and implementation notes 2

2 Figures and tables **3**

3 Citations and further reading **3**
3.1 References 4

1 Purpose

This specimen tests the shared visual language for the course notes before we write the first full lecture. It includes a beautiful external link, internal cross-references, mathematics, source code, a figure, a table, callouts, and a scholarly citation.

The design should remain quiet enough for sustained reading while making definitions, examples, and implementation details easy to find.

Learning objectives

After reading a course note, students should know what they are expected to understand, implement, and explain—not merely which pages they have completed.

1.1 Mathematical typography

A language model assigns a probability to a token sequence. Using the chain rule,

$$P(w_{1:n}) = \prod_{i=1}^{n} P(w_i \mid w_{1:i-1}). \tag{1}$$

Equation 1 should be readable inline with the surrounding explanation. For a bigram approximation,

$$P(w_i \mid w_{1:i-1}) \approx P(w_i \mid w_{i-1}).$$

> **Intuition**
>
> The chain rule is exact. The approximation appears only when we limit how much history the model may use.

1.2 Code and implementation notes

Code should fit the page, preserve indentation, and remain visually distinct from prose.

```python
from collections import Counter

def unigram_distribution(tokens: list[str]) -> dict[str, float]:
    """Estimate a maximum-likelihood unigram distribution."""
    counts = Counter(tokens)
    total = sum(counts.values())
    return {token: count / total for token, count in counts.items()}
```

> **Common mistake**
>
> A maximum-likelihood estimate assigns zero probability to every unseen token. Later notes will introduce smoothing and explicit unknown-token handling.

2 Figures and tables

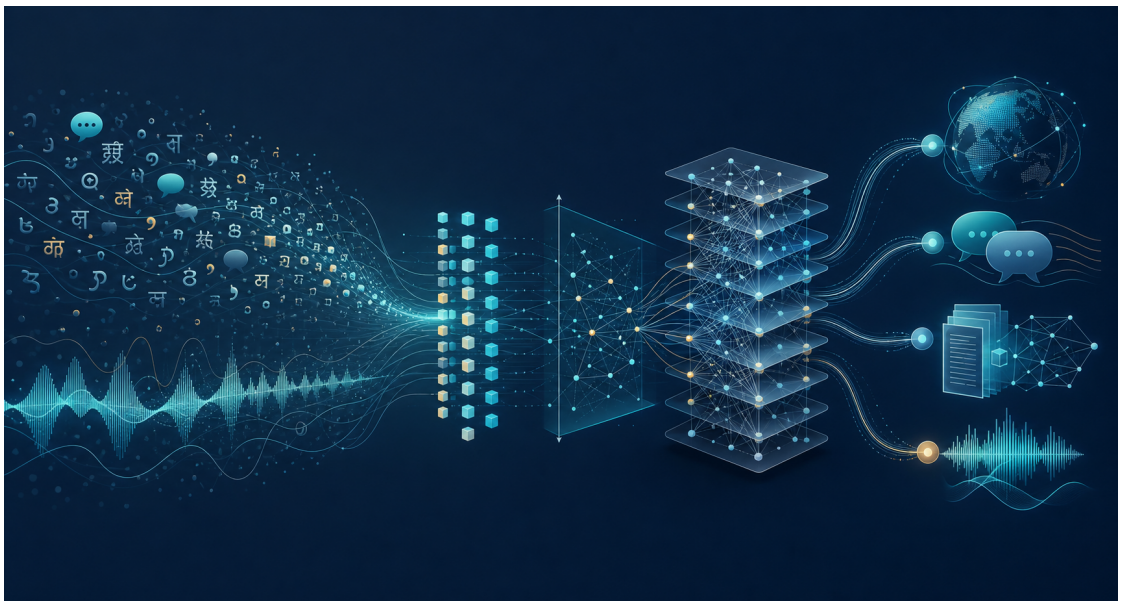


Figure 1: The course-wide view of language flowing through representations and models.

As illustrated in Figure 1, modern NLP connects multiple input modalities, representations, and downstream tasks.

Table 1: A compact comparison of common text representations.

Representation	Dimensionality	Context sensitive	Typical era
One-hot vector	Vocabulary size	No	Symbolic/statistical
Static embedding	50–300	No	Neural representations
Contextual embedding	Model width	Yes	Transformers

Table 1 demonstrates the preferred table style: minimal rules, aligned values, and a meaningful caption.

> **!** **Checkpoint**
>
> Before continuing, a student should be able to explain why the vector for *bank* is fixed in a static embedding but changes across sentences in a contextual model.

3 Citations and further reading

Course notes will cite primary sources where they clarify the origin of an idea. The Transformer architecture, for example, was introduced by Vaswani et al. (2017).

3.1 References

Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. “Attention Is All You Need.” *Advances in Neural Information Processing Systems* 30. https://arxiv.org/abs/1706.03762.